

Python Programming

Unit 02 – Lecture 05: Advanced Functions and Functional Patterns

Tofik Ali

School of Computer Science, UPES Dehradun

February 20, 2026

Repository: <https://github.com/tali7c/Python-Programming>

Quick Links

[Core Concepts](#)[Demo](#)[Interactive](#)[Summary](#)

Agenda

- 1 Overview
- 2 Core Concepts
- 3 Demo
- 4 Interactive
- 5 Summary

- Differentiate common function types and choose by use case

Learning Outcomes

- Differentiate common function types and choose by use case
- Pass collections safely and reason about mutation

Learning Outcomes

- Differentiate common function types and choose by use case
- Pass collections safely and reason about mutation
- Apply recursion with a correct base case

Learning Outcomes

- Differentiate common function types and choose by use case
- Pass collections safely and reason about mutation
- Apply recursion with a correct base case
- Use `map`, `filter`, and `lambda`

Learning Outcomes

- Differentiate common function types and choose by use case
- Pass collections safely and reason about mutation
- Apply recursion with a correct base case
- Use `map`, `filter`, and `lambda`
- Recognize side effects in functions

Functions Are Objects (First-class)

- You can store a function in a variable

```
def square(x):  
    return x * x  
  
f = square  
print(f(5))    # 25
```

Functions Are Objects (First-class)

- You can store a function in a variable
- You can pass a function into another function (*higher-order*)

```
def square(x):  
    return x * x  
  
f = square  
print(f(5))    # 25
```

Functions Are Objects (First-class)

- You can store a function in a variable
- You can pass a function into another function (*higher-order*)
- You can return a function from a function (closure / factory)

```
def square(x):  
    return x * x  
  
f = square  
print(f(5))    # 25
```

Function Types and Typical Use Cases

Type	When you typically use it
Built-in	fast + readable (<code>len</code> , <code>sum</code> , <code>sorted</code>)
User-defined (<code>def</code>)	reuse and name logic; easiest to test
Anonymous (<code>lambda</code>)	small one-off function for <code>key=</code> / <code>map</code>
Higher-order	take/return functions (customizable pipelines)
Generator (<code>yield</code>)	produce values lazily for large/streaming data
Recursive	natural fit for trees / nested structures (use carefully)

Predicate Functions (for filter)

- A predicate returns True/False

```
marks = [95, None, 120, 70]
```

```
def is_valid(x):  
    return (  
        isinstance(x, int)  
        and 0 <= x <= 100  
    )
```

```
valid = list(filter(is_valid, marks))
```

Predicate Functions (for filter)

- A predicate returns True/False
- Use it to keep only valid items

```
marks = [95, None, 120, 70]
```

```
def is_valid(x):  
    return (  
        isinstance(x, int)  
        and 0 <= x <= 100  
    )
```

```
valid = list(filter(is_valid, marks))
```

Key Functions (for sorted/min/max)

- A key function returns a value used for comparison

```
students = [  
    ("A", 9.1),  
    ("B", 8.3),  
    ("C", 9.4),  
]  
  
top = max(students, key=lambda t: t[1])
```

Key Functions (for sorted/min/max)

- A key function returns a value used for comparison
- Use it to sort/rank complex records (tuples, dicts, objects)

```
students = [  
    ("A", 9.1),  
    ("B", 8.3),  
    ("C", 9.4),  
]
```

```
top = max(students, key=lambda t: t[1])
```


Passing Collections

- Lists and dicts are mutable

```
def add_item(lst, item):  
    lst.append(item)
```

Passing Collections

- Lists and dicts are mutable
- Functions can modify them in place

```
def add_item(lst, item):  
    lst.append(item)
```

Passing Collections

- Lists and dicts are mutable
- Functions can modify them in place
- Use copies when you want to preserve input

```
def add_item(lst, item):  
    lst.append(item)
```

Worked Example: Avoid Unintended Mutation

```
def sort_copy(items):  
    tmp = items.copy()  
    tmp.sort()  
    return tmp  
  
data = [3, 1, 2]  
print(sort_copy(data))    # [1, 2, 3]  
print(data)               # [3, 1, 2]
```

- Copy when you want a function without side effects

Recursion Basics

- A function calls itself

Recursion Basics

- A function calls itself
- Needs a base case to stop

Recursion Basics

- A function calls itself
- Needs a base case to stop
- Each call reduces the problem size

Recursive Example

```
def factorial(n):  
    if n == 1:  
        return 1  
    return n * factorial(n - 1)
```


Pitfall: Recursion in Python

- Missing base case leads to infinite recursion

Pitfall: Recursion in Python

- Missing base case leads to infinite recursion
- Deep recursion can hit recursion limits

Pitfall: Recursion in Python

- Missing base case leads to infinite recursion
- Deep recursion can hit recursion limits
- Prefer iteration for simple loops in Python

map and filter

- map applies a function to each item

```
nums = [1, 2, 3, 4]
print(list(map(lambda x: x*x, nums)))
print(list(filter(lambda x: x%2==0, nums)))
```

map and filter

- map applies a function to each item
- filter keeps items that satisfy a condition

```
nums = [1, 2, 3, 4]
print(list(map(lambda x: x*x, nums)))
print(list(filter(lambda x: x%2==0, nums)))
```

Pitfall: map/filter Return Iterators

- In Python 3, `map` and `filter` return iterators

Pitfall: map/filter Return Iterators

- In Python 3, `map` and `filter` return iterators
- Wrap with `list(...)` to materialize results

Generator Functions (yield)

- Generates values one-by-one (lazy) instead of building a full list

```
def count_up_to(n):  
    i = 1  
    while i <= n:  
        yield i  
        i += 1  
  
print(list(count_up_to(5)))
```


Generator Functions (yield)

- Generates values one-by-one (lazy) instead of building a full list
- Saves memory and works well for large/streaming data

```
def count_up_to(n):  
    i = 1  
    while i <= n:  
        yield i  
        i += 1  
  
print(list(count_up_to(5)))
```

Generator Functions (yield)

- Generates values one-by-one (lazy) instead of building a full list
- Saves memory and works well for large/streaming data
- Often used in pipelines and file processing

```
def count_up_to(n):  
    i = 1  
    while i <= n:  
        yield i  
        i += 1  
  
print(list(count_up_to(5)))
```

Lambda and Inner Functions

- lambda creates small anonymous functions

```
def make_multiplier(k):  
    def mult(x):  
        return x * k  
    return mult
```

Lambda and Inner Functions

- lambda creates small anonymous functions
- Inner functions can capture variables (closures)

```
def make_multiplier(k):  
    def mult(x):  
        return x * k  
    return mult
```

Lambda and Inner Functions

- lambda creates small anonymous functions
- Inner functions can capture variables (closures)
- Inner functions can hide helper logic (encapsulation)

```
def make_multiplier(k):  
    def mult(x):  
        return x * k  
    return mult
```

Encapsulation Example (Helper Inside a Function)

```
def normalize_marks(marks):  
    def clip(x):  
        return max(0, min(x, 100))  
    return [clip(m) for m in marks]
```

- clip is used internally and not exposed outside

Encapsulation Example (Helper Inside a Function)

```
def normalize_marks(marks):  
    def clip(x):  
        return max(0, min(x, 100))  
    return [clip(m) for m in marks]
```

- clip is used internally and not exposed outside
- Keeps implementation details private and focused

Side Effects vs Pure Functions

- Pure functions do not change external state

Side Effects vs Pure Functions

- Pure functions do not change external state
- Side effects can make debugging harder

Side Effects vs Pure Functions

- Pure functions do not change external state
- Side effects can make debugging harder
- Prefer pure functions when possible

Demo: Cleaning Data with map/filter

- Remove invalid values

Script: `demo/functional_tools.py`

Demo: Cleaning Data with map/filter

- Remove invalid values
- Transform items with map

Script: `demo/functional_tools.py`

Demo: Cleaning Data with map/filter

- Remove invalid values
- Transform items with `map`
- Compute recursive sum

Script: `demo/functional_tools.py`

Demo: Cleaning Data with map/filter

- Remove invalid values
- Transform items with `map`
- Compute recursive sum
- Extension: rewrite using comprehensions and compare readability

Script: `demo/functional_tools.py`

Checkpoint 1

Identify the base case in a recursive function. Why is it necessary?

Checkpoint 2

Replace this loop with map:

```
nums = [1,2,3,4,5]
squares = []
for n in nums:
    squares.append(n*n)
```


Think-Pair-Share

When should you avoid recursion in Python? Provide one reason and one example.

Summary

- Passing collections can mutate inputs

Summary

- Passing collections can mutate inputs
- Recursion needs a base case

Summary

- Passing collections can mutate inputs
- Recursion needs a base case
- `map/filter/lambda` support functional style

Summary

- Passing collections can mutate inputs
- Recursion needs a base case
- `map/filter/lambda` support functional style
- Prefer pure functions for clarity and testing

Exit Question

If a function appends to a list passed as argument, does the caller see the change? Why?